МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение высшего образования
«Российский экономический университет имени Г.В. Плеханова»

Пермский институт (филиал)

Сопроводительная записка

по производственной практике

к программному модулю «Импорт товаров в электронный магазин из CSV-файлов»

ПП.02.01 Технология разработки программного обеспечения
(индекс по РУП и наименование практики)

Профессионального модуля ПМ.02 Осуществление интеграции программных модулей
(индекс по РУП и наименование профессионального модуля)

Специальность 09.02.07 Информационные системы и программирование

Квалификация программист

Обучающийся _____ Кононов Даниил Павлович
(подпись) (фамилия, имя, отчество)

Группа ИПо-41

Руководитель практической подготовки от института

_____ Берестов Дмитрий Борисович
(подпись) (фамилия, имя, отчество)

«13» ноября 2026 года

Пермь, 2026 год

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 4
1. ЦЕЛЬ И ЗАДАЧИ ПРАКТИКИ 5
2. ОПИСАНИЕ ЗАДАНИЯ 6
2.1. Назначение модуля импорта 6
2.2. Функциональные требования 6
2.3. Требования к качеству и ограничения 6
2.4. Вариант индивидуализации «Одежда» 7
2.5. Процесс до и после внедрения модуля 7
3. ПРОЕКТИРОВАНИЕ РЕШЕНИЯ 9
3.1. Инструменты и технологии 9
3.2. Структура проекта 9
3.3. Назначение компонентов 10
3.4. Схема работы модуля 11
3.5. Ход разработки и работа с Git 12
4. ФОРМАТ CSV-ФАЙЛА 14
4.1. Поля файла 14
4.2. Правила проверки данных 14
4.3. Пример файла 15
5. РЕАЛИЗАЦИЯ МОДУЛЯ 17
5.1. Модели данных 17
5.2. Хранение товаров в памяти 17
5.3. Разбор строк CSV 18
5.4. Чтение файла и загрузка через браузер 19
5.5. Проверка данных 20
5.6. Обработка дубликатов артикула 21
5.7. Контроллер и передача результата в представление 22
5.8. Пользовательский интерфейс 22
6. ТЕСТИРОВАНИЕ И ОТЛАДКА 25
6.1. Тестовые сценарии 25

6.2. Модульные тесты 26
6.3. Отладка и контроль качества кода 27
7. РУКОВОДСТВО ПО ЗАПУСКУ И РАБОТЕ 29
ЗАКЛЮЧЕНИЕ 30

ВВЕДЕНИЕ

Современный интернет-магазин невозможно представить без быстрого наполнения каталога товарами. Когда ассортимент насчитывает сотни или тысячи позиций, вносить их вручную по одной неудобно и долго, а вероятность ошибок при таком вводе очень высока. Типичное решение этой проблемы заключается в загрузке данных из файла, который поставщик или сотрудник магазина готовит в табличном редакторе и сохраняет в формате CSV.

В рамках производственной практики ПП.02.01 было получено индивидуальное задание разработать модуль импорта товаров из CSV-файла в веб-приложение на платформе ASP.NET Core MVC. Модуль читает файл, проверяет каждую строку, добавляет корректные товары в память приложения и показывает пользователю результат: сколько строк обработано, сколько товаров загружено и какие ошибки найдены с указанием номеров строк.

Для индивидуализации задания выбран вариант 3 «Одежда». Он добавляет к базовой модели товара поля size (размер), color (цвет) и material (материал) и требует отдельных правил проверки для них. Приложение получило название WearStore.

В записке описан ход работы: анализ задания, проектирование структуры приложения, формат входного файла, реализация каждого компонента, тестирование и отладка. Исходный код хранится в системе контроля версий Git, а работоспособность модуля подтверждена модульными тестами и ручным тестированием через браузер.

1. ЦЕЛЬ И ЗАДАЧИ ПРАКТИКИ

Целью практики является формирование практических умений по разработке и интеграции программного модуля в веб-приложение: от анализа требований до отладки, тестирования и оформления документации. Задание направлено на освоение профессиональных компетенций ПК 2.1–2.5: разработку требований к программному модулю, интеграцию модуля в приложение, его отладку, разработку тестовых сценариев и проверку кода на соответствие стандартам.

Для достижения цели необходимо было решить следующие задачи:

– Изучить требования к модулю импорта товаров и описать формат CSV-файла.

– Спроектировать модель товара с дополнительными полями варианта «Одежда» и структуру хранения данных в памяти.

– Реализовать чтение CSV-файла с поддержкой пустых полей и полей в кавычках.

– Реализовать проверку обязательных полей и допустимости значений, формирование списка ошибок с номерами строк.

– Загрузить корректные товары в коллекцию в памяти и обработать повторяющиеся артикулы.

– Создать интерфейс на ASP.NET Core MVC для запуска импорта и просмотра списка товаров.

– Подготовить модульные тесты и тестовые сценарии, выполнить отладку модуля.

– Провести самопроверку кода на соответствие базовым стандартам.

– Разместить проект в репозитории Git и оформить настоящую записку.

2. ОПИСАНИЕ ЗАДАНИЯ

2.1. Назначение модуля импорта

Модуль предназначен для массового наполнения электронного магазина товарами. Пользователь указывает путь к CSV-файлу или выбирает файл на своём компьютере. Приложение читает его построчно, преобразует строки в объекты C#, проверяет данные и сохраняет корректные товары в памяти. Результат импорта отображается на странице: общее число строк, число успешно загруженных товаров, число ошибок и таблица ошибок с номерами строк.

Вторая часть модуля, страница списка товаров, показывает всё, что было загружено. Это позволяет сразу убедиться, что данные из файла прочитаны верно.

2.2. Функциональные требования

По заданию модуль должен выполнять следующие действия:

- загружать CSV-файл с данными о товарах;
- читать данные из файла построчно;
- преобразовывать строки CSV в объекты C#;
- проверять корректность данных;
- добавлять корректные товары в коллекцию в памяти;
- показывать результат импорта и список ошибок;
- отображать список товаров, загруженных из файла.

База данных в задании не предусмотрена: товары хранятся только в памяти приложения, и после его перезапуска список очищается. Это допустимо по условиям задания.

2.3. Требования к качеству и ограничения

Код должен быть читаемым, а ответственность между компонентами разделена: контроллер не содержит сложной логики, импорт выполняется в отдельном сервисе, хранение данных отделено от представления. Приложение

не должно падать из-за некорректного файла. Должны обрабатываться случаи: файл не найден, файл пуст, неверные заголовки, неверная цена, отрицательный остаток, дубликат артикула, пустое название товара. Импорт можно запускать повторно без сбоев.

Ограничениями реализации являются хранение данных только в памяти и кодировка входных файлов UTF-8. Код размещается в Git с осмысленными сообщениями коммитов.

2.4. Вариант индивидуализации «Одежда»

Базовая логика импорта одинакова для всех вариантов, а предметная область меняет дополнительные поля. Для варианта «Одежда» добавлены три поля, описанные в таблице 1.

Таблица 1 – Дополнительные поля варианта «Одежда»

Поле	**Описание**	**Правило проверки**
size	Размер	Буквенный размер XS, S, M, L, XL, XXL или целое число от 36 до 60
color	Цвет	Обязательно
material	Материал	Может быть пустым

Задание допускает «числовой размер», но не уточняет диапазон. Для российской размерной сетки выбраны значения от 36 до 60, а границы вынесены в константы, чтобы их можно было легко изменить.

2.5. Процесс до и после внедрения модуля

До появления модуля новые товары пришлось бы вносить в систему вручную: открывать форму, заполнять поля одного товара, сохранять и повторять эти действия для каждой позиции. При таком подходе легко допустить опечатку в цене, пропустить обязательное поле или создать товар с уже существующим артикулом, а найти ошибку после ввода затруднительно.

После внедрения модуля сотрудник готовит таблицу в привычном редакторе, сохраняет её в CSV и загружает одним действием. Модуль сам проверяет все строки и сообщает, какие из них приняты, а какие отклонены и по

какой причине. Ошибки видны сразу, с номером строки, поэтому исправить файл и повторить загрузку можно за несколько минут. Время наполнения каталога сокращается с часов ручной работы до секунд обработки, а риск пропустить неверные данные снижается за счёт автоматической проверки.

3. ПРОЕКТИРОВАНИЕ РЕШЕНИЯ

3.1. Инструменты и технологии

Модуль разработан на языке C# в среде Visual Studio 2022 на платформе .NET 8. Для веб-части использован фреймворк ASP.NET Core MVC, который разделяет приложение на модели, представления и контроллеры. Внешний вид страниц построен на библиотеке Bootstrap, входящей в стандартный шаблон проекта. Для модульного тестирования применён фреймворк xUnit, для контроля версий используется Git, а репозиторий размещён на сервисе GitHub.

CSV-файлы читаются без сторонних библиотек: для разбора строк написан собственный парсер. Это позволило полностью контролировать обработку кавычек, пустых полей и запятых внутри значений, а также лучше понять, как устроен формат.

3.2. Структура проекта

Решение WearStore состоит из двух проектов: основного веб-приложения WearStore и тестового проекта WearStore.Tests. В основном проекте код разделён по папкам в соответствии с ролью файлов: в папке Models лежат модели данных и константы, в Services находится вся логика работы с данными, в Controllers расположен контроллер, а в Views хранятся страницы. Файлы с тестовыми данными размещены в папке wwwroot/csv (основной пример) и в папке testdata тестового проекта.

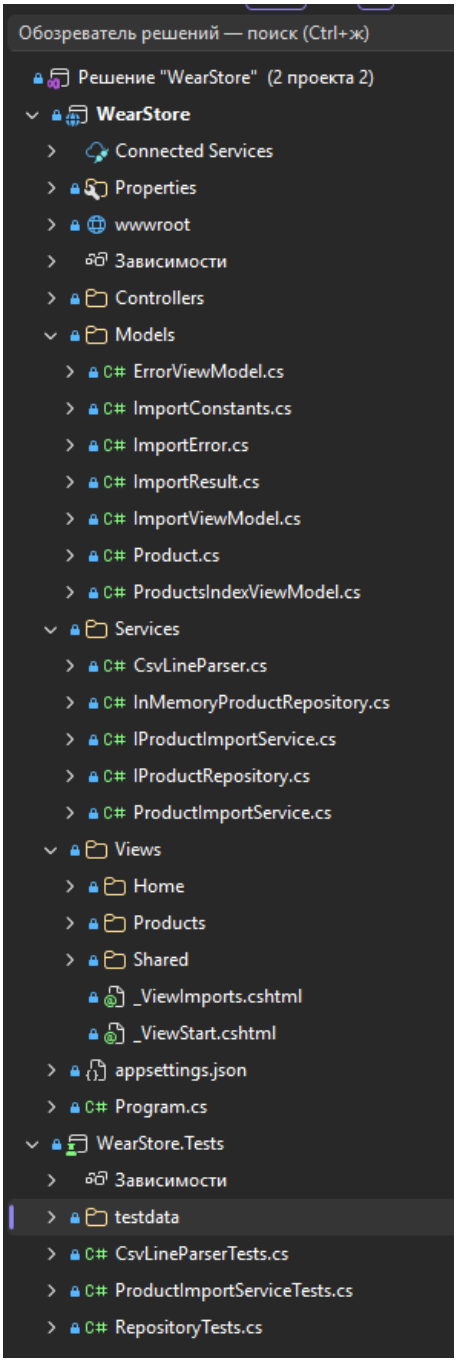


Рисунок 1 - Структура проекта WearStore

3.3. Назначение компонентов

Каждый компонент решает одну задачу, поэтому код легко читать и проверять по частям.

– Product - модель товара. Содержит базовые поля (артикул, название, описание, цена, валюта, категория, остаток, ссылка на изображение, признак активности) и поля варианта «Одежда» (размер, цвет, материал).

– ImportResult и ImportError - результат импорта. Хранят число обработанных строк, число загруженных товаров и список ошибок, в каждой из

которых указан номер строки и текст сообщения.

– ImportConstants - константы: валюта и категория по умолчанию, допустимые буквенные размеры, границы числового размера.

– IProductRepository и InMemoryProductRepository - хранилище товаров в памяти. Интерфейс описывает операции, а класс реализует их на основе списка List.

– CsvLineParser - разбор одной строки CSV на поля.

– IProductImportService и ProductImportService - сервис импорта: чтение, проверка данных, добавление товаров в репозиторий и формирование результата.

– ProductsController - контроллер. Принимает запросы, вызывает сервис и передаёт результат в представление, не содержит логики импорта.

– Представления Index и Import - страницы со списком товаров и с формой импорта.

Репозиторий и сервис зарегистрированы в приложении как singleton, то есть существуют в единственном экземпляре на всё время работы. Это важно: если бы репозиторий создавался заново на каждый запрос, товары, загруженные на странице импорта, пропадали бы при переходе на страницу списка.

3.4. Схема работы модуля

Работа модуля происходит в несколько этапов. Пользователь выбирает CSV-файл на странице импорта. Контроллер принимает запрос и передаёт файл сервису импорта. Сервис читает строки, с помощью парсера разбивает их на поля, проверяет каждое поле и превращает корректные строки в объекты Product. Готовые товары добавляются в репозиторий, а ошибки собираются в список. В конце сервис возвращает объект ImportResult, который контроллер передаёт в представление. Страница списка товаров получает данные напрямую из репозитория.

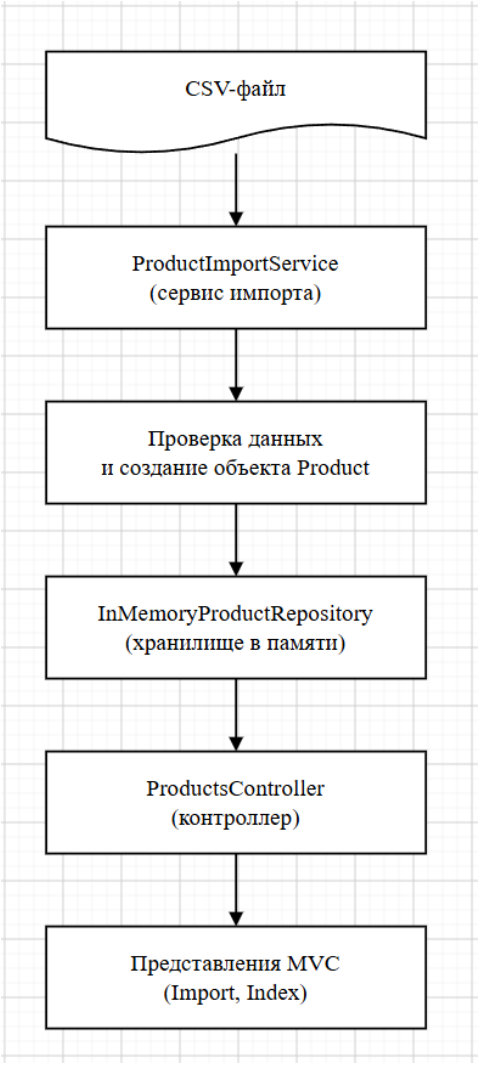


Рисунок 2 - Схема работы модуля импорта

3.5. Ход разработки и работа с Git

Разработка шла поэтапно, и после каждого законченного этапа создавался коммит с понятным сообщением. Сначала были созданы проект и модели данных, затем репозиторий в памяти и тесты для него. Следующим этапом стал парсер CSV-строк, после него сервис импорта с проверкой данных и тестами. Затем были добавлены контроллер и страницы, загрузка файла через браузер, очистка списка, поиск, фильтр по категории и сортировка. Последними созданы тестовые CSV-файлы и описание проекта в файле README.md.

Такой порядок позволил проверять каждую часть сразу после написания и не накапливать ошибки. Весь код хранится в репозитории на GitHub, а история коммитов показывает последовательность работы.

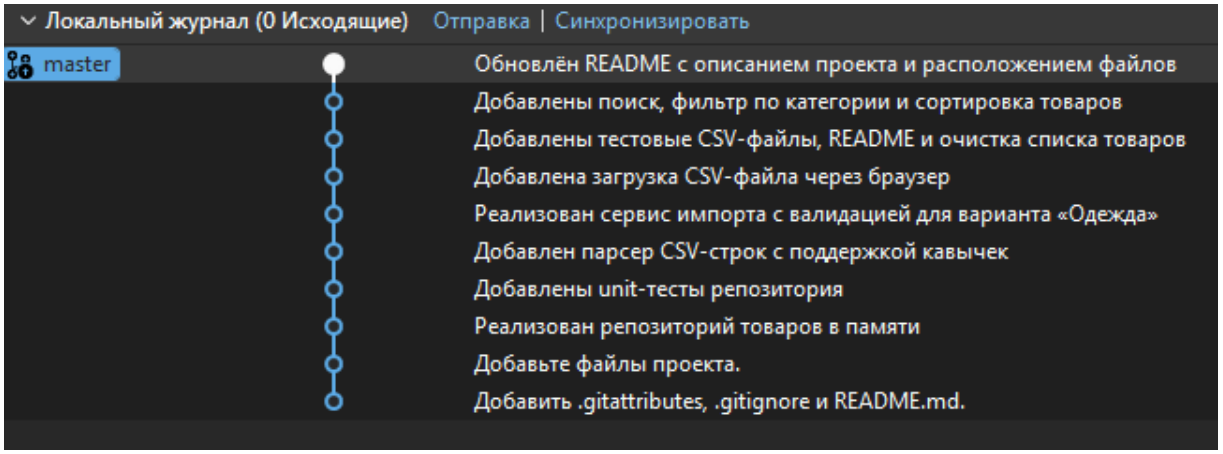


Рисунок 3 - История коммитов проекта

4. ФОРМАТ CSV-ФАЙЛА

4.1. Поля файла

Импортируемый файл должен быть в кодировке UTF-8 и содержать в первой непустой строке заголовки колонок. Значения разделяются запятой. Если значение содержит запятую, его нужно заключить в двойные кавычки, а кавычки внутри значения удваиваются. Названия колонок сравниваются без учёта регистра и пробелов по краям. Дополнительные колонки, не входящие в список, игнорируются, а при отсутствии любой из обязательных импорт отклоняется. Поля файла перечислены в таблице 2.

Таблица 2 – Поля CSV-файла

Поле	**Тип**	**Обязательность**	**Описание**
sku	строка	обязательно	Артикул товара, уникальное значение
name	строка	обязательно	Название товара
description	строка	нет	Описание
price	число	обязательно	Цена, больше 0
currency	строка	нет	Валюта, по умолчанию RUB
category_name	строка	нет	Категория, по умолчанию «Без категории»
stock	целое число	обязательно	Остаток на складе, не меньше 0
image_url	строка	нет	Ссылка на изображение
is_active	логическое	обязательно	Активность: true, false, 1 или 0
size	строка	обязательно	Размер товара
color	строка	обязательно	Цвет
material	строка	нет	Материал

4.2. Правила проверки данных

Каждая строка проходит проверку по всем полям, и если в ней найдено несколько ошибок, пользователь увидит их все сразу. Благодаря этому не нужно загружать файл несколько раз, исправляя ошибки по одной. Правила и сообщения собраны в таблице 3.

Таблица 3 – Правила проверки и сообщения об ошибках

Поле	**Правило**	**Сообщение об ошибке**
sku	не пустое	Поле sku не может быть пустым
name	не пустое	Поле name не может быть пустым
price	число больше 0	Поле price должно быть числом больше 0
stock	целое число не меньше 0	Остаток на складе должен быть целым числом не меньше 0
is_active	true, false, 1 или 0	Поле is_active должно иметь значение true, false, 1 или 0
size	XS, S, M, L, XL, XXL или число от 36 до 60	Размер должен быть XS, S, M, L, XL, XXL или числом от 36 до 60
color	не пустое	Поле color обязательно для варианта «Одежда»

Кроме проверки полей, выполняются проверки всего файла и строки. Если файл не найден, выводится сообщение «Файл не найден». Если файл пустой или в нём нет строк данных, выводится «Данные для импорта отсутствуют». Если в заголовке нет обязательных колонок, выводится «Неверные заголовки CSV». Если в строке не совпадает число полей с заголовком, выводится «Неверное количество полей в строке». Если артикул уже есть в памяти, строка пропускается с сообщением «Артикул товара уже существует».

Номер строки в сообщении равен номеру физической строки файла: заголовок имеет номер 1, первая строка с товаром номер 2. Пустые строки пропускаются и в число обработанных не входят.

4.3. Пример файла

Основной пример хранится в проекте по пути wwwroot/csv/sample_products.csv. В нём 11 корректных товаров разных категорий: часть размеров указана буквами, часть числами, у одного товара пустой материал, а у одного название содержит запятую и поэтому взято в кавычки.

Листинг 1 - Фрагмент файла sample_products.csv

```
sku,name,description,price,currency,category_name,stock,image_url,is_active,size
,color,material 3001,Футболка белая,Летняя футболка из
хлопка,990,RUB,Футболки,25,/images/tshirt.jpg,true,M,белый,хлопок 3002,Джинсы
прямые синие,Классические
джинсы,3490,RUB,Джинсы,12,/images/jeans.jpg,true,46,синий,деним
```

5. РЕАЛИЗАЦИЯ МОДУЛЯ

5.1. Модели данных

Основной моделью является класс Product, который описывает один товар магазина. В нём собраны базовые поля из задания и три дополнительных поля варианта «Одежда»: размер, цвет и материал. Все текстовые поля по умолчанию инициализируются пустой строкой, поэтому в объекте не появляются неопределённые значения. Валюта и категория получают значения по умолчанию из класса констант ImportConstants.

Листинг 2 - Основные поля класса Product

```
public class Product
{
    public string Sku { get; set; } = string.Empty;
    public string Name { get; set; } = string.Empty;
    public decimal Price { get; set; }
    public int StockQuantity { get; set; }
    public bool IsActive { get; set; }
    public string Size { get; set; } = string.Empty;
    public string Color { get; set; } = string.Empty;
    public string Material { get; set; } = string.Empty;
}
```

Результат работы импорта описывают классы ImportResult и ImportError. ImportResult хранит число обработанных строк, число загруженных товаров и список ошибок, а число ошибок вычисляется как длина этого списка. ImportError содержит номер строки и текст сообщения. Для ошибок, не связанных с конкретной строкой (например, «Файл не найден»), используется номер 0.

5.2. Хранение товаров в памяти

Товары хранятся в классе InMemoryProductRepository внутри обычного списка List. Репозиторий реализует интерфейс IProductRepository, поэтому остальной код зависит от интерфейса, а не от конкретного класса, и хранилище можно заменить, например, на базу данных без изменения сервиса и

контроллера.

Так как репозиторий существует в единственном экземпляре и к нему могут обращаться несколько запросов одновременно, все операции выполняются внутри блокировки lock. Метод GetAll возвращает копию списка, чтобы внешний код не мог испортить внутренние данные. Поиск по артикулу не зависит от регистра и игнорирует пробелы по краям.

Листинг 3 - Добавление товара и поиск по артикулу

```
public void Add(Product product)
{
    if (product == null)
        throw new ArgumentNullException(nameof(product));

    lock (_lock)
    {
        _products.Add(product);
    }
}
public Product? GetBySku(string sku)
{
    if (string.IsNullOrWhiteSpace(sku))
        return null;

    lock (_lock)
    {
        return _products.FirstOrDefault(p =>
            string.Equals(p.Sku, sku.Trim(),
StringComparison.OrdinalIgnoreCase));
    }
}
```

5.3. Разбор строк CSV

Строку CSV нельзя просто разделить по запятым: внутри значения запятая может быть частью текста, например в названии «Футболка, оверсайз». Поэтому для разбора написан класс CsvLineParser. Он проходит строку по символам и запоминает, находится ли сейчас внутри кавычек. Запятая вне кавычек завершает поле, а запятая внутри кавычек становится частью значения. Две двойные кавычки подряд внутри кавычек превращаются в одну, так что значение «Футболка “”Special””» будет прочитано как Футболка “Special”.

Пустые поля тоже обрабатываются корректно: две запятые подряд дают пустое значение, и число полей в строке остаётся равным числу колонок. После разбора у каждого поля убираются пробелы по краям, поэтому лишние пробелы в файле не влияют на результат.

Листинг 4 - Обработка кавычек в парсере строк

```
if (inQuotes)
{
    if (c == '"')
    {
        if (i + 1 < line.Length && line[i + 1] == '"')
        {
            current.Append('"');
            i++;
        }
        else
        {
            inQuotes = false;
        }
    }
    else
    {
        current.Append(c);
    }
}
```

5.4. Чтение файла и загрузка через браузер

Сервис импорта умеет получать данные тремя способами, и все они сходятся в одном методе ImportFromLines, где находится основная логика. Метод ImportFromCsv принимает путь к файлу: сначала проверяет, что путь указан и файл существует, затем читает все строки. Чтение обёрнуто в блок try-catch для исключений IOException и UnauthorizedAccessException, поэтому недоступный или занятый другой программой файл не приводит к падению приложения, а превращается в понятное сообщение для пользователя.

Листинг 5 - Проверка существования и чтение файла

```
if (string.IsNullOrWhiteSpace(filePath) || !File.Exists(filePath))
{
    AddError(result, 0, FileNotFoundMessage);
    return result;
```

```
}

string[] lines;
try
{
    lines = File.ReadAllLines(filePath);
}
catch (Exception ex) when (ex is IOException || ex is
UnauthorizedAccessException)
{
    AddError(result, 0, FileReadErrorMessage);
    return result;
}
```

Метод ImportFromStream служит для загрузки файла через браузер. Контроллер получает от формы загруженный файл и передаёт сервису его поток, а сервис читает поток построчно в кодировке UTF-8. Чтение файла остаётся обязанностью сервиса, а контроллер лишь передаёт данные дальше, как требует разделение ответственности. Третий способ, метод ImportFromLines, принимает уже готовые строки. Он нужен для модульных тестов, чтобы проверять логику импорта без создания файлов на диске.

5.5. Проверка данных

Проверка выполняется в методе ParseProduct сервиса импорта. Для каждой строки по очереди проверяются поля sku, name, price, stock, is_active, size и color. Главная особенность в том, что проверка не останавливается на первой найденной ошибке: все ошибки строки собираются в общий список, и пользователь видит их сразу. Если хотя бы одна ошибка найдена, товар в память не добавляется.

Числовые значения разбираются с учётом особенностей записи. В цене допускается как точка, так и запятая в качестве десятичного разделителя, а при разборе используется инвариантная культура, поэтому результат не зависит от региональных настроек компьютера. Логическое поле is_active принимает значения true и false без учёта регистра, а также 1 и 0.

Листинг 6 - Проверка цены, остатка и размера

```
if (!TryParseDecimal(Get("price"), out decimal price) || price <= 0)
    Fail(InvalidPriceMessage);

if (!TryParseInt(Get("stock"), out int stock) || stock < 0)
    Fail(InvalidStockMessage);

if (!TryNormalizeSize(Get("size"), out string size))
    Fail(InvalidSizeMessage);
```

Размер проверяется отдельным методом TryNormalizeSize. Сначала значение сравнивается со списком буквенных размеров без учёта регистра, и если совпадение найдено, размер приводится к верхнему регистру: значение m станет M. Если буквенного размера нет, значение разбирается как целое число и проверяется на попадание в диапазон от 36 до 60. Границы диапазона и список букв хранятся в классе ImportConstants, поэтому в коде нет «магических чисел».

Необязательные поля получают значения по умолчанию. Пустая валюта заменяется на RUB, а указанная приводится к верхнему регистру. Пустая категория заменяется на «Без категории». Описание, ссылка на изображение и материал могут быть пустыми.

5.6. Обработка дубликатов артикула

Артикул товара должен быть уникальным. Поведение при повторе по заданию нужно выбрать из трёх вариантов: пропустить дубликат, обновить существующий товар или добавить как новый с предупреждением. Выбран рекомендованный вариант: дубликат пропускается, а в список ошибок добавляется сообщение «Артикул товара уже существует» с номером строки.

Проверка на дубликат выполняется только после того, как строка прошла все остальные проверки, чтобы не показывать пользователю лишних сообщений. Для поиска используется репозиторий. Поскольку каждый успешно импортированный товар сразу попадает в репозиторий, одна и та же проверка находит как товары, загруженные ранее, так и повторы внутри текущего файла. В репозитории при этом остаётся первый товар, а второй пропускается.

Листинг 7 - Проверка дубликата и добавление товара

```
if (_repository.GetBySku(product.Sku) != null)
```

```
{
    AddError(result, rowNumber, DuplicateSkuMessage);
    continue;
}

_repository.Add(product);
result.ImportedCount++;
```

Благодаря такой логике повторный импорт того же файла не приводит к сбою и не создаёт копий товаров: все строки отклоняются как дубликаты, а список товаров остаётся прежним.

5.7. Контроллер и передача результата в представление

Контроллер ProductsController получает репозиторий и сервис импорта через конструктор. Он содержит три основных действия: Index показывает список товаров, Import (GET) открывает форму, а Import (POST) запускает импорт. Логики проверки данных в контроллере нет: он только определяет путь к файлу, вызывает сервис и передаёт результат в представление.

Путь можно указать относительно папки wwwroot (например, /csv/sample_products.csv) или полностью (например, C:\data\products.csv). Для загрузки файла через браузер используется отдельное действие ImportUpload, а для очистки списка товаров действие Clear.

Листинг 8 - Запуск импорта в контроллере

```
var result = _importService.ImportFromCsv(resolvedPath);

return View(new ImportViewModel
{
    FilePath = filePath ?? string.Empty,
    Result = result
});
```

5.8. Пользовательский интерфейс

Интерфейс состоит из двух страниц, доступных из меню сверху. На странице «Импорт» есть два способа выбрать файл: ввести путь или выбрать файл на компьютере. После запуска показывается таблица итогов (всего строк,

импортировано, ошибки) и таблица ошибок с номерами строк.

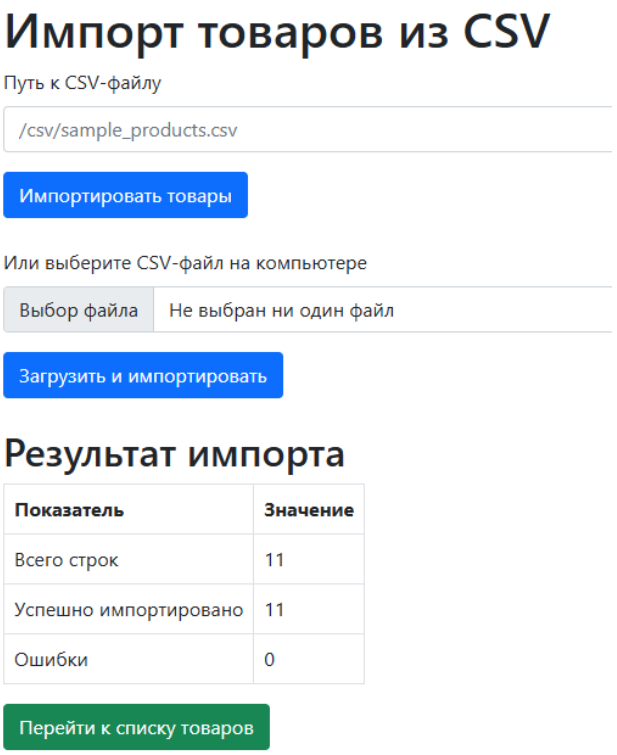


Рисунок 4 - Результат успешного импорта

Страница «Товары» показывает таблицу со всеми полями, включая размер, цвет и материал, а также количество товаров и их общую стоимость.

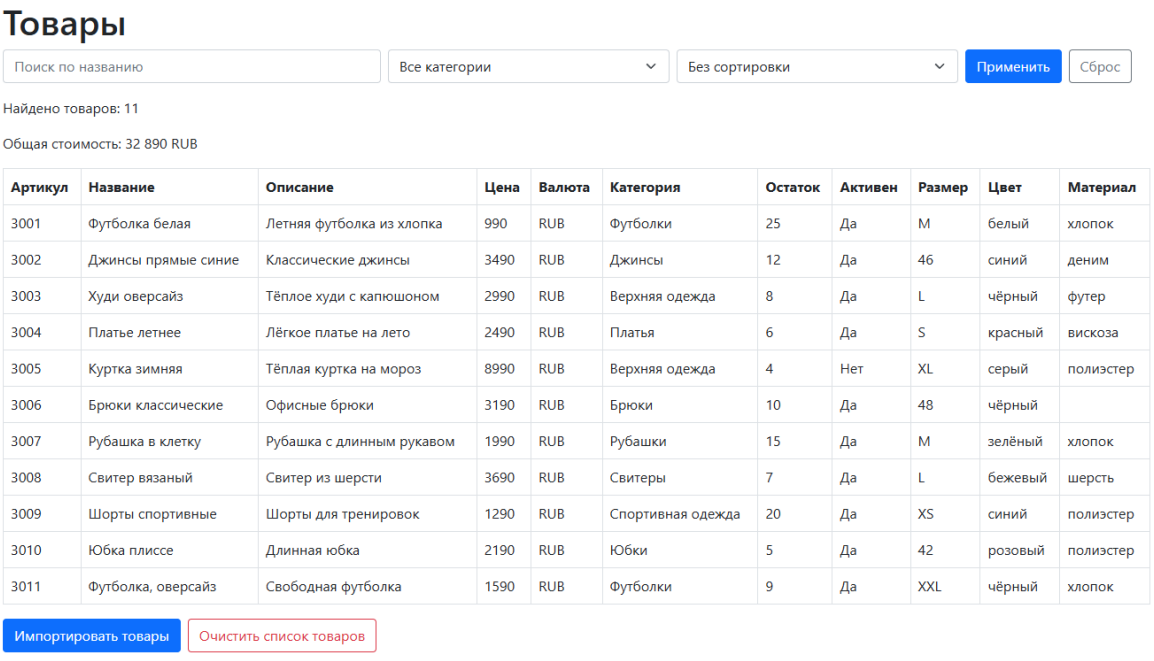
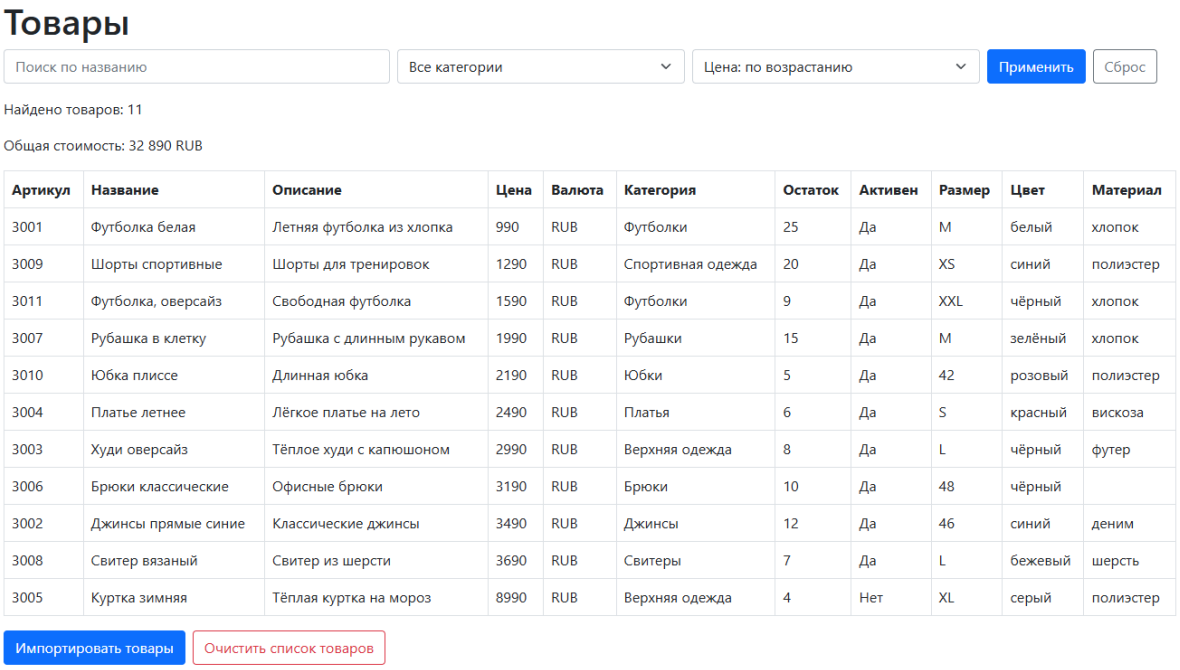


Рисунок 5 - Список товаров

Дополнительно реализованы поиск по названию, фильтр по категории и сортировка по цене или остатку, а также кнопка очистки списка.



Артикул	Название	Описание	Цена	Валюта	Категория	Остаток	Активен	Размер	Цвет	Материал
3001	Футболка белая	Летняя футболка из хлопка	990	RUB	Футболки	25	Да	M	белый	хлопок
3009	Шорты спортивные	Шорты для тренировок	1290	RUB	Спортивная одежда	20	Да	XS	синий	полиэстер
3011	Футболка, оверсайз	Свободная футболка	1590	RUB	Футболки	9	Да	XXL	чёрный	хлопок
3007	Рубашка в клетку	Рубашка с длинным рукавом	1990	RUB	Рубашки	15	Да	M	зелёный	хлопок
3010	Юбка плиссе	Длинная юбка	2190	RUB	Юбки	5	Да	42	розовый	полиэстер
3004	Платье летнее	Лёгкое платье на лето	2490	RUB	Платья	6	Да	S	красный	вискоза
3003	Худи оверсайз	Тёплое худи с капюшоном	2990	RUB	Верхняя одежда	8	Да	L	чёрный	футер
3006	Брюки классические	Офисные брюки	3190	RUB	Брюки	10	Да	48	чёрный	
3002	Джинсы прямые синие	Классические джинсы	3490	RUB	Джинсы	12	Да	46	синий	деним
3008	Свитер вязаный	Свитер из шерсти	3690	RUB	Свитеры	7	Да	L	бежевый	шерсть
3005	Куртка зимняя	Тёплая куртка на мороз	8990	RUB	Верхняя одежда	4	Нет	XL	серый	полиэстер

Рисунок 6 - Поиск, фильтр и сортировка товаров

6. ТЕСТИРОВАНИЕ И ОТЛАДКА

6.1. Тестовые сценарии

Работа модуля проверялась вручную через браузер на подготовленных CSV-файлах. Каждый файл проверяет одну ситуацию: корректные данные или конкретную ошибку. Файлы лежат в папке testdata тестового проекта, а основной пример в wwwroot/csv. Перед каждой проверкой список товаров очищался кнопкой «Очистить список товаров», чтобы данные прошлых загрузок не влияли на результат. Сценарии и результаты собраны в таблице 4.

Таблица 4 - Результаты тестовых сценариев

№	Сценарий	Файл	Ожидаемый результат	Фактический результат
1	Успешный импорт	valid_clothing.csv	Импортировано 2 из 2, ошибок нет	Выполнено
2	Неверная цена	invalid_price.csv	Ошибка в строке 2: поле price должно быть числом больше 0	Выполнено
3	Дубликат артикула	duplicate_sku.csv	Импортировано 1 из 2, ошибка в строке 3: артикул товара уже существует	Выполнено
4	Отрицательный остаток	negative_stock.csv	Ошибка: остаток на складе должен быть целым числом не меньше 0	Выполнено
5	Неверный размер	bad_size.csv	Ошибка: размер должен быть XS, S, M, L, XL, XXL или числом от 36 до 60	Выполнено
6	Цвет не указан	missing_color.csv	Ошибка: поле color обязательно для варианта «Одежда»	Выполнено

№	Сценарий	Файл	Ожидаемый результат	Фактический результат
7	Неверное значение is_active	bad_active.csv	Ошибка: поле is_active должно иметь значение true, false, 1 или 0	Выполнено
8	Пустой файл	empty_file.csv	Сообщение «Данные для импорта отсутствуют», приложение не падает	Выполнено
9	Кириллица	sample_products.csv	Импортировано 11 из 11, русский текст отображается корректно	Выполнено

Во всех случаях приложение не падало, а пользователь получал понятное сообщение.

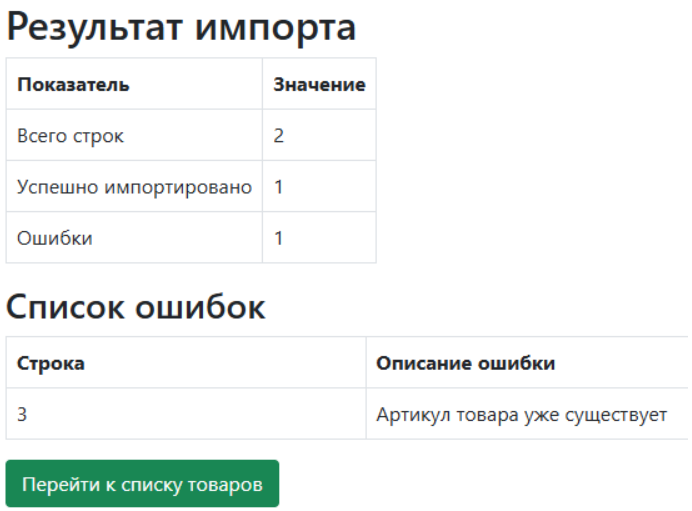


Рисунок 7 - Обработка дубликата артикула

6.2. Модульные тесты

Для проверки логики без браузера написаны 36 модульных тестов на фреймворке xUnit. Они разделены на три группы:

– тесты репозитория (6): добавление, поиск по артикулу без учёта регистра и пробелов, пустой репозиторий, копия списка в GetAll, очистка;

– тесты парсера (5): обычная строка, пустые поля, запятая внутри кавычек, экранированные кавычки, обрезка пробелов;

– тесты сервиса импорта (25): успешный импорт, ошибка для каждого поля, неверные заголовки, неверное число полей, дубликаты, несколько ошибок в одной строке, значения по умолчанию, кириллица и загрузка из потока.

Сервис тестируется через метод ImportFromLines, поэтому тесты не зависят от файловой системы и выполняются за доли секунды. Все 36 тестов проходят успешно.

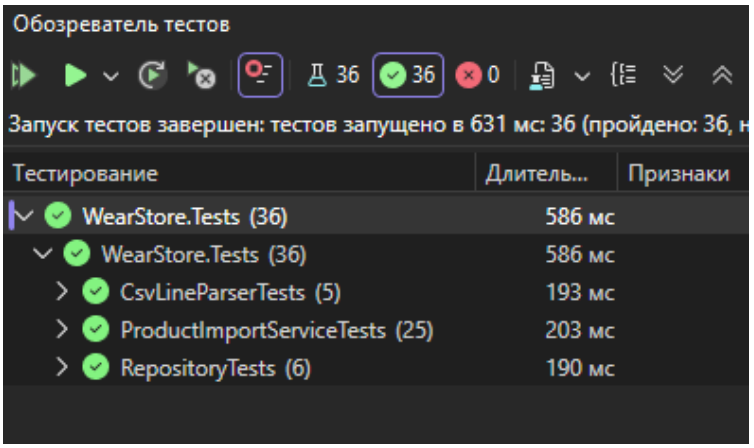


Рисунок 8 - Результаты модульных тестов

6.3. Отладка и контроль качества кода

Отладка выполнялась в Visual Studio на тестовых файлах и модульных тестах. В ходе проверки были найдены и учтены особенности работы приложения.

При повторной загрузке файла sample_products.csv все 11 строк были отклонены как дубликаты. Это подтвердило, что репозиторий сохраняет данные между запросами, а проверка артикулов работает верно. Для удобства тестирования на страницу списка добавлена кнопка очистки списка товаров.

При разборе пути к файлу в первой версии использовалась проверка Path.IsPathRooted. На Windows она считает корневым и путь вида /csv/sample_products.csv, поэтому файл из папки wwwroot не находился.

Проверка заменена на Path.IsPathFullyQualified, и теперь одинаково работают оба вида путей: относительный от wwwroot и полный.

Файлы, сохранённые в UTF-8 с сигнатурой, начинаются с невидимого символа BOM. Чтобы он не портил название первой колонки, при построении карты заголовков этот символ удаляется.

Самопроверка кода по критериям задания приведена в таблице 5.

Таблица 5 – Результаты самопроверки кода

Критерий	**Как выполнено**
Имена классов	Понятные названия: ProductImportService, InMemoryProductRepository, CsvLineParser
Имена методов	Отражают действие: ImportFromCsv, ImportFromLines, GetBySku, ParseProduct
Обработка ошибок	Неверный файл не роняет приложение, все ошибки попадают в ImportResult
Разделение ответственности	Контроллер вызывает сервис, чтение и проверка данных выполняются в сервисе
Магические числа	Валюта, категория по умолчанию, размеры и их границы вынесены в ImportConstants, тексты сообщений в константы сервиса
Исключения	IOException и UnauthorizedAccessException перехватываются при чтении файла
Читаемость и комментарии	Код отформатирован, ключевые методы снабжены краткими комментариями
Git-коммиты	Осмысленные сообщения на русском языке

7. РУКОВОДСТВО ПО ЗАПУСКУ И РАБОТЕ

Для запуска приложения нужна Visual Studio 2022 с рабочей нагрузкой «ASP.NET и разработка веб-приложений» и .NET 8. Порядок работы такой:

1. Скачать проект из репозитория GitHub и открыть файл решения WearStore.sln.
2. Нажать F5. Откроется браузер со страницей «Товары», пока пустой.
3. Перейти в раздел «Импорт» и указать путь /csv/sample_products.csv или выбрать любой CSV-файл на компьютере кнопкой «Выберите файл».
4. Нажать «Импортировать товары» или «Загрузить и импортировать» и посмотреть результат.
5. Перейти на страницу «Товары» и проверить загруженные данные, при необходимости воспользоваться поиском, фильтром и сортировкой.
6. Для повторной проверки нажать «Очистить список товаров».

ЗАКЛЮЧЕНИЕ

В ходе производственной практики ПП.02.01 разработан модуль импорта товаров из CSV-файла для веб-приложения WearStore на ASP.NET Core MVC по варианту 3 «Одежда». Модуль читает файл, проверяет каждую строку, добавляет корректные товары в память приложения и показывает результат с номерами строк для найденных ошибок. Все требования задания выполнены: есть страницы импорта и списка товаров, обработка ошибок, хранение данных в памяти, модульные тесты и проект в системе контроля версий Git. Дополнительно реализованы загрузка файла через браузер, поиск, фильтр по категории, сортировка и очистка списка товаров.

В процессе работы были получены практические навыки: разработка требований к модулю, проектирование структуры приложения, работа с файлами CSV, использование коллекций в памяти, интеграция модуля в веб-приложение, отладка в Visual Studio, написание модульных тестов на xUnit и работа с Git.

Основные трудности были связаны с разбором CSV и путями к файлам. Запятые и кавычки внутри значений потребовали собственного парсера, а неверное определение относительного пути к файлу было найдено при отладке и исправлено заменой проверки пути. Проблему повторных артикулов при повторной загрузке решила единая проверка через репозиторий.

В дальнейшем модуль можно улучшить: подключить базу данных вместо хранения в памяти, добавить поддержку других разделителей и кодировок CSV (например, Windows-1251), выполнять импорт асинхронно, добавить пагинацию списка товаров и экспорт товаров в CSV.